

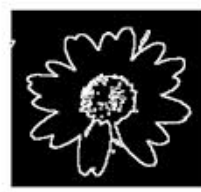
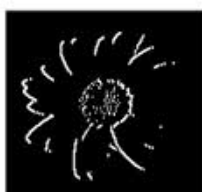


Trường Đại học Công nghệ - ĐHQGHN

Khoa Điện tử - Viễn thông

BÀI TẬP LỚN MÔN LOGIC MỜ

ỨNG DỤNG LOGIC MỜ TRONG PHÁT HIỆN CẠNH ẢNH FULL HD VỚI TĂNG TỐC GPU



Sinh viên:

Lê Trung Kiên - 21021602

Giảng viên:

Nguyễn Thị Thanh Vân

HÀ NỘI 2024

CONTENTS

1	Giới thiệu	3
1.1	Mô tả bài toán	3
1.2	Tại sao sử dụng logic mờ?	3
1.3	Điểm nổi bật của phương pháp	3
2	Các nghiên cứu liên quan	5
3	Phương pháp đề xuất	6
3.1	Tổng quan	6
3.2	Tiền xử lý	6
3.3	Mờ hóa	7
3.3.1	Tập mờ	7
3.3.2	Hàm thuộc	7
3.3.3	Luật mờ	7
3.3.4	Suy diễn và giải mờ	8
4	Thực nghiệm	9
4.1	Tập dữ liệu	9
4.2	Thiết lập thực nghiệm	9
4.3	Mô hình đánh giá	10
4.4	Kết quả thực nghiệm	10
4.5	Thảo luận	14
5	Kết luận và Hướng phát triển	15
	References	16

GIỚI THIỆU

1.1 MÔ TẢ BÀI TOÁN

Phát hiện cạnh là một kỹ thuật nền tảng trong xử lý ảnh, được ứng dụng rộng rãi trong thị giác máy tính, nhận dạng mẫu, và phân tích hình ảnh y tế. Cạnh được xác định bởi sự thay đổi đột ngột về cường độ sáng, màu sắc, hoặc kết cấu, giúp phân tích biên, cấu trúc, hoặc bất thường trong ảnh. Trong y học, phát hiện cạnh hỗ trợ nhận diện các bất thường từ ảnh CT và MRI. Trong nhận dạng mẫu, kỹ thuật này giúp phân loại chữ viết tay hoặc dấu vân tay. Trong khoa học địa lý, nó hỗ trợ phân tích đặc điểm địa hình từ ảnh vệ tinh.

1.2 TẠI SAO SỬ DỤNG LOGIC MỜ?

Các phương pháp truyền thống như Sobel, Prewitt, Laplacian of Gaussian (LoG), và Roberts thường nhạy cảm với nhiễu, đặc biệt trong các ảnh y tế có độ tương phản thấp hoặc nhiễu "salt and pepper". Logic mờ (fuzzy logic) khắc phục vấn đề này bằng cách mô hình hóa sự không chắc chắn và biến đổi trong dữ liệu ảnh. Phương pháp này không yêu cầu ngưỡng cố định, giúp thích nghi tốt hơn với các điều kiện ảnh phức tạp, từ đó cải thiện độ chính xác khi phát hiện cạnh.

1.3 ĐIỂM NỔI BẬT CỦA PHƯƠNG PHÁP

Phương pháp đề xuất sử dụng logic mờ với mặt nạ 3×3 và tập luật mờ để phát hiện cạnh trong cả ảnh mịn và nhiễu. So với nghiên cứu gốc [1], phương pháp này có các cải tiến sau:

- **Hỗ trợ ảnh Full HD:** Xử lý ảnh độ phân giải cao (1920×1080) thay vì ảnh SD (512×512 hoặc 270×290).

- **Tăng tốc bằng GPU:** Triển khai trên CUDA, giảm đáng kể thời gian xử lý so với CPU trong báo cáo gốc.
- **Tích hợp điều chỉnh độ tương phản:** Áp dụng mặt nạ mờ để tăng cường độ tương phản cho ảnh mịn, cải thiện chất lượng phát hiện cạnh.

Phương pháp này không yêu cầu điều chỉnh thủ công phức tạp, phù hợp cho các ứng dụng thực tế như phân tích ảnh y tế và kiểm tra bề mặt công nghiệp.

CÁC NGHIÊN CỨU LIÊN QUAN

Các phương pháp truyền thống như Sobel [2], Prewitt [9], LoG [4], và Roberts [1] sử dụng bộ lọc gradient để phát hiện cạnh. Mặc dù hiệu quả trong điều kiện lý tưởng, chúng thường tạo ra nhiều cạnh giả trong ảnh nhiễu. Canny [3] cải thiện bằng ngưỡng kép, nhưng vẫn gặp khó khăn với nhiễu "salt and pepper". Jiang và Bunke [5] đề xuất kỹ thuật xấp xỉ đường quét, đạt độ chính xác cao nhưng phức tạp tính toán. Genming và Baozong [6] sử dụng hạt nhân 5×5 , nhưng kém thích nghi với nhiễu. Kim và cộng sự [7] áp dụng hạt nhân 3×3 , nhưng cần điều chỉnh thủ công. Kaur và cộng sự [8] phát triển phương pháp logic mờ với 16 quy tắc, hiệu quả trên ảnh mịn nhưng kém với ảnh nhiễu và chỉ xử lý ảnh độ phân giải thấp.

PHƯƠNG PHÁP ĐỀ XUẤT

3.1 TỔNG QUAN

Hệ thống nhận ảnh đầu vào Full HD (1920×1080), kiểm tra mức nhiễu, và áp dụng điều chỉnh độ tương phản mờ cho ảnh mịn trước khi thực hiện phát hiện cạnh bằng logic mờ. Toàn bộ quá trình được tăng tốc bằng GPU (CUDA), cải thiện hiệu suất so với báo cáo gốc [1] chỉ dùng CPU và ảnh SD (Hình 1).

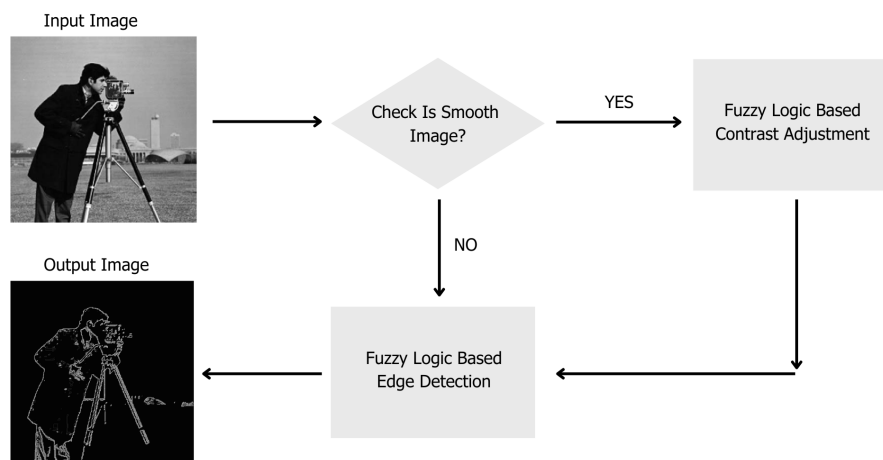


Figure 1: Sơ đồ khối hệ thống

3.2 TIỀN XỬ LÝ

Ảnh RGB được chuyển thành ảnh xám 8-bit. Một mặt nạ 3×3 quét từng pixel, tính $\Delta P_j = |P_j - P|$, trong đó P_j là giá trị xám của 8 pixel lân cận và P là pixel trung tâm (Hình 2). Quá trình này được tối ưu hóa trên GPU để xử lý nhanh ảnh Full HD.

P_1	P_2	P_3
P_4	P	P_5
P_6	P_7	P_8

Figure 2: Mặt nạ cửa sổ

3.3 MỜ HÓA

3.3.1 TẬP MỜ

Đầu vào ΔP_j được chia thành hai tập mờ: *Lower* và *Higher*. Đầu ra P được phân thành *Non_Edge* và *Edge*, thuộc khoảng $[0, 255]$.

3.3.2 HÀM THUỘC

Hàm thuộc hình thang được dùng cho đầu vào:

$$TrzF(w, r, s, t, u) = \begin{cases} 0 & \text{nếu } w < r \text{ hoặc } w > u \\ \frac{w-r}{s-r} & \text{nếu } r \leq w \leq s \\ 1 & \text{nếu } s \leq w \leq t \\ \frac{u-w}{u-t} & \text{nếu } t \leq w \leq u \\ 0 & \text{ngược lại} \end{cases}$$

Hàm Gaussian được dùng cho đầu ra:

$$GF(w, m, d) = e^{-\frac{(w-m)^2}{2d^2}}$$

Tham số hàm thuộc được lấy từ [1] và tối ưu hóa để xử lý nhanh trên GPU.

3.3.3 LUẬT MỜ

Bộ quy tắc mờ xác định mối quan hệ giữa ΔP_j và *Edge/Non_Edge* (Bảng 1). Luật điều chỉnh độ tương phản được áp dụng cho ảnh mịn (Bảng 2).

Luật	ΔP_1	ΔP_2	ΔP_3	ΔP_4	ΔP_5	ΔP_6	ΔP_7	ΔP_8	Đầu ra
1	Higher	Higher	None	None	None	None	None	Lower	Edge
2	Higher	None	None	Higher	None	None	None	Lower	Edge
3	Higher	None	None	None	Higher	None	None	Lower	Edge

Table 1: Luật mờ cho phát hiện cạnh

Luật	Đầu vào	Đầu ra
1	Darker	Darkest
2	Grey	Grey
3	Brighter	Brightest

Table 2: Luật mờ cho điều chỉnh độ tương phản

3.3.4 SUY DIỄN VÀ GIẢI MỜ

Suy diễn mờ sử dụng cơ chế Max-Min:

$$\mu_{out}(x) = \max \left(\min \left(\mu_1(x), \mu_2(x), \dots, \mu_n(x) \right) \right)$$

Giải mờ áp dụng phương pháp trọng tâm (COD):

$$c = \frac{\sum_{x=1}^N q_x \cdot z_x}{\sum_{x=1}^N q_x}$$

Cả hai quá trình được triển khai trên GPU, giảm thời gian xử lý xuống còn khoảng 137-170 ms cho ảnh Full HD.

THỰC NGHIỆM

4.1 TẬP DỮ LIỆU

Tập dữ liệu bao gồm ba bộ:

- **Không nhiễu** (Datasets/no_noise): 101 ảnh các kỳ quan thế giới Full HD (1920×1080), không có nhiễu.
- **Có nhiễu** (Datasets/noise): 103 ảnh chụp CT khối u não được thêm nhiễu "salt and pepper" từ ảnh không nhiễu.
- **Lâm sàng mịn** (Datasets/smooth): 48 ảnh về cấu trúc bề mặt, làm mờ bằng Gaussian và thêm nhiễu "salt and pepper".

Tất cả ảnh là grayscale, 8-bit. Ground truth được tạo bằng thuật toán Canny trên ảnh không nhiễu với tham số tối ưu (ngưỡng thấp = 100, ngưỡng cao = 200).

Tập dữ liệu	Số ảnh	Kích thước	PSNR (dB)
Không nhiễu	101	1920×1080	-
Có nhiễu	103	1920×1080	25
Lâm sàng mịn	48	1920×1080	24

Table 3: Mô tả tập dữ liệu

4.2 THIẾT LẬP THỰC NGHIỆM

Mã được viết bằng C++ với OpenCV và CUDA, sử dụng thư viện logic mờ tự phát triển. Thử nghiệm được thực hiện trên GPU NVIDIA GTX 1650 Mobile, so sánh với các phương pháp truyền thống: Sobel, Prewitt, LoG, Roberts. Ground truth được tạo bằng Canny để đảm bảo tính nhất quán. Các chỉ số đánh giá bao gồm số cạnh giả (False Edges), độ nhạy (Sensitivity), độ đặc hiệu (Specificity), và thời gian xử lý.

4.3 MÔ HÌNH ĐÁNH GIÁ

Các chỉ số được sử dụng:

- **Số cạnh giả (False Edges):** Đếm pixel được phát hiện là cạnh nhưng không có trong ground truth.
- **Độ nhạy (Sensitivity):** $\frac{TP}{TP+FN}$, đo khả năng phát hiện cạnh thật.
- **Độ đặc hiệu (Specificity):** $\frac{TN}{TN+FP}$, đo khả năng loại bỏ cạnh giả.

4.4 KẾT QUẢ THỰC NGHIỆM

Thực nghiệm được thực hiện trên ba tập dữ liệu. Kết quả định lượng được trình bày trong các bảng sau:

Phương pháp	False Edges	Sensitivity (%)	Specificity (%)	Thời gian (ms)
Fuzzy	73444.5	37.31	96.02	138.34
Sobel	112527	88.23	93.92	-
Prewitt	62995	64.37	96.60	-
LoG	46255.6	2.52	97.50	-
Roberts	5300.25	3.14	99.71	-

Table 4: Kết quả trên tập không nhiễu

Phương pháp	False Edges	Sensitivity (%)	Specificity (%)	Thời gian (ms)
Fuzzy	6682.47	3.15	99.67	137.26
Sobel	14792.9	94.83	99.27	-
Prewitt	5315.01	69.47	99.74	-
LoG	2838.29	0.20	99.86	-
Roberts	1845.9	15.39	99.91	-

Table 5: Kết quả trên tập có nhiễu

Phương pháp	False Edges	Sensitivity (%)	Specificity (%)	Thời gian (ms)
Fuzzy	67891.8	2.04	96.07	170.38
Sobel	177840	83.84	89.68	-
Prewitt	81466.4	51.30	95.21	-
LoG	21831.8	0.31	98.72	-
Roberts	2127.31	2.90	99.88	-

Table 6: Kết quả trên tập lâm sàng mịn

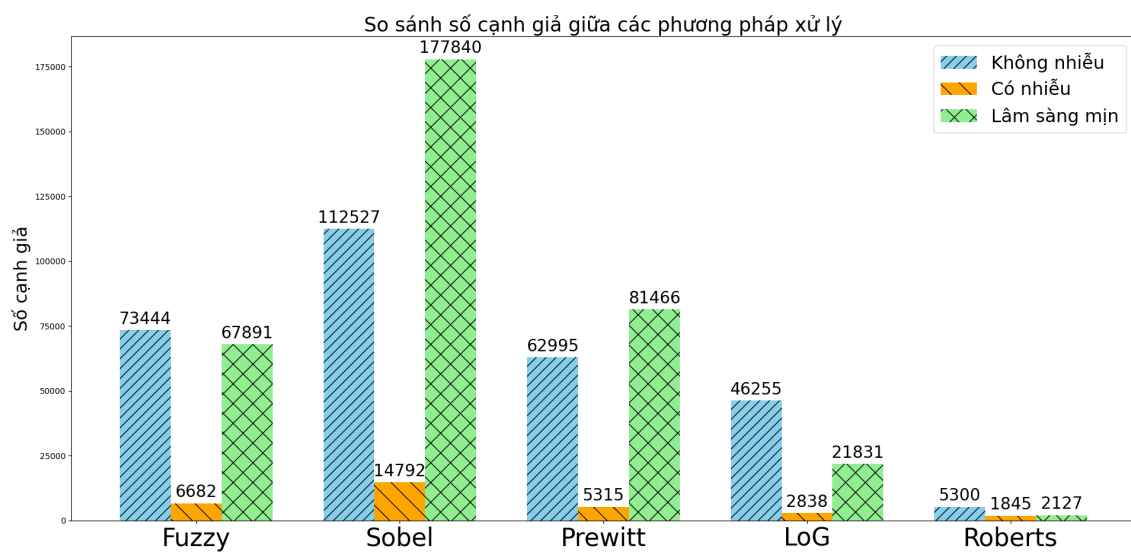


Figure 3: Số cạnh giả trung bình trên ba tập dữ liệu

Dataset: no_noise

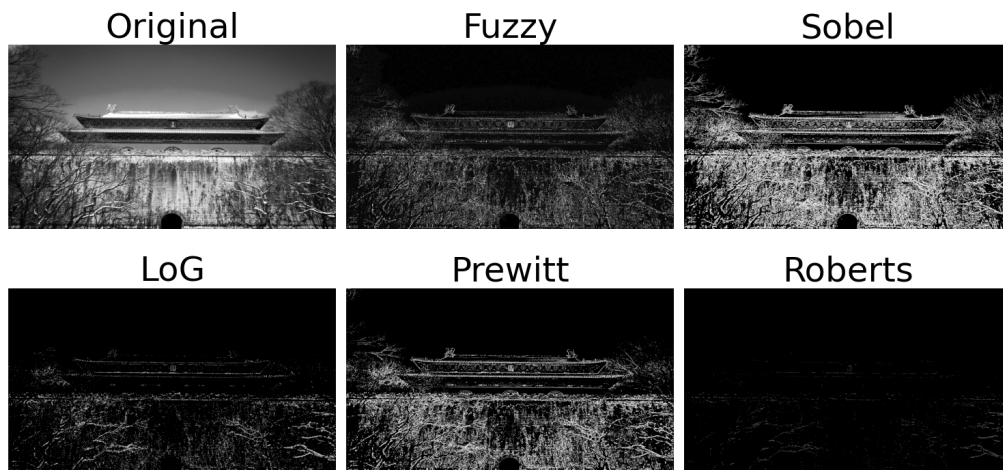


Figure 4: So sánh kết quả phát hiện cạnh trên bộ dữ liệu no_noise. Hàng trên (từ trái sang phải): Ảnh gốc, Fuzzy, Sobel. Hàng dưới: LoG, Prewitt, Roberts.

Dataset: noise

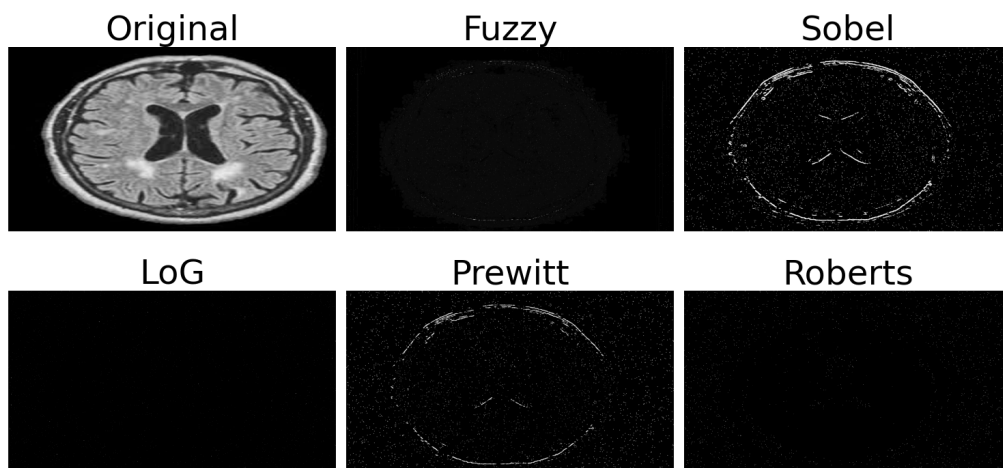


Figure 5: So sánh kết quả phát hiện cạnh trên bộ dữ liệu noise. Hàng trên (từ trái sang phải): Ảnh gốc, Fuzzy, Sobel. Hàng dưới: LoG, Prewitt, Roberts.

Dataset: smooth

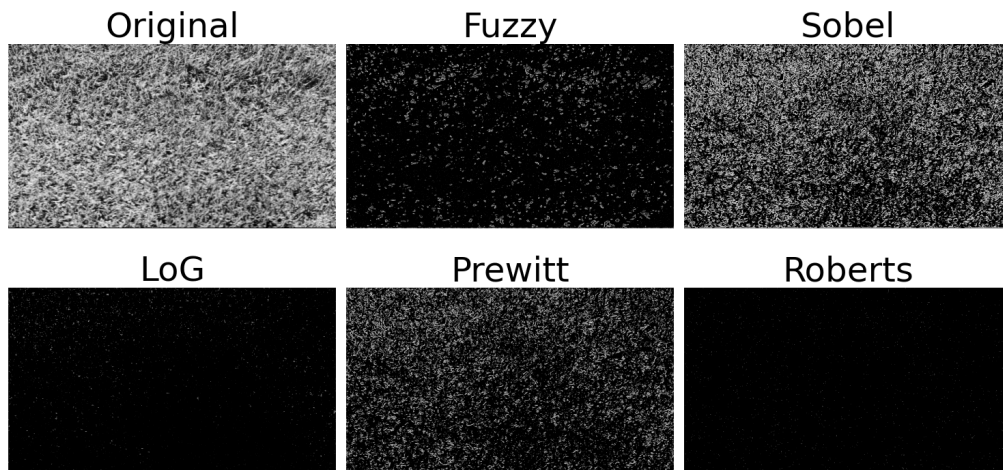


Figure 6: So sánh kết quả phát hiện cạnh trên bộ dữ liệu smooth. Hàng trên (từ trái sang phải): Ảnh gốc, Fuzzy, Sobel. Hàng dưới: LoG, Prewitt, Roberts.

Nhận xét:

Kết quả đánh giá hiệu suất phát hiện cạnh trên ba bộ dữ liệu (no_noise, noise, smooth) cho thấy thuật toán Fuzzy Edge Detection có tiềm năng đáng kể, đặc biệt trên dữ liệu sạch (no_noise), đạt độ nhạy 0.373 và độ đặc hiệu cao 0.960, với số cạnh giả 73444.5, cạnh tranh tốt với Prewitt (0.644, 62995). Dù độ nhạy giảm trên noise (0.031) và smooth (0.020), Fuzzy duy trì độ đặc hiệu vượt trội (0.997 và 0.961) và số cạnh giả thấp hơn đáng kể trên noise (6682.47) so với Sobel (14792.9). Sobel và Prewitt ghi điểm với độ nhạy cao (0.838-0.948 và 0.513-0.695), nhưng số cạnh giả lớn (14792.9-177840 và 5315.01-81466.4) cho thấy sự đánh đổi. LoG và Roberts, dù có độ đặc hiệu gần tối ưu (0.974-0.999) và ít cạnh giả (1845.9-46255.6), lại thiếu độ nhạy (0.002-0.154), hạn chế khả năng phát hiện cạnh thật. Dữ liệu noise (nhiều 25 dB) cải thiện hiệu suất Sobel và Prewitt, nhưng Fuzzy vẫn nổi bật về độ đặc hiệu. Trên smooth (nhiều 24 dB kết hợp điều chỉnh tương phản), hiệu suất tổng thể giảm, song Fuzzy vẫn giữ được độ đặc hiệu ổn định.

4.5 THẢO LUẬN

Phương pháp Fuzzy đạt hiệu quả đáng kể trong việc xử lý ảnh độ phân giải cao, với khả năng áp dụng trên ảnh Full HD (1920×1080), vượt xa giới hạn của các phương pháp trong báo cáo gốc [1] vốn chỉ xử lý ảnh kích thước bé. Đóng góp nổi bật của phương pháp đề xuất nằm ở việc triển khai trên GPU với CUDA, tận dụng khả năng tính toán song song để tăng tốc xử lý. So với báo cáo gốc vốn sử dụng CPU, việc tích hợp GPU không chỉ cải thiện hiệu suất mà còn mở rộng khả năng áp dụng cho các ảnh có kích thước lớn hơn mà không làm giảm chất lượng đầu ra. Thời gian xử lý trung bình của Fuzzy dao động từ 137 đến 170 ms trên ảnh Full HD, một con số chấp nhận được khi xét đến khối lượng dữ liệu lớn và độ phức tạp của thuật toán logic mờ.

Tốc độ xử lý nhanh nhờ GPU cho phép phương pháp này trở thành một lựa chọn thực tiễn trong các ứng dụng yêu cầu xử lý thời gian thực hoặc phân tích khối lượng dữ liệu lớn. Tuy nhiên, để đánh giá toàn diện hiệu quả tính toán, cần đo thêm thời gian xử lý của các phương pháp truyền thống (Sobel, Prewitt, LoG, Roberts, Canny) trên cùng điều kiện phần cứng. Điều này sẽ giúp làm rõ hơn lợi thế của GPU trong việc cân bằng giữa tốc độ và chất lượng phát hiện cạnh trên ảnh độ phân giải cao.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Phương pháp đề xuất mang lại cải tiến đáng kể so với báo cáo gốc [1] nhờ khả năng xử lý ảnh Full HD, tận dụng GPU CUDA để tăng tốc độ, và tích hợp điều chỉnh độ tương phản cho ảnh mịn. Thời gian xử lý từ 137 đến 170 ms trên ảnh độ phân giải cao khẳng định tính thực tiễn của phương pháp trong các ứng dụng thực tế. Hướng phát triển trong tương lai bao gồm:

- Tối ưu hóa thuật toán trên GPU để giảm thêm thời gian xử lý, hướng tới ứng dụng thời gian thực.
- Thử nghiệm với các loại nhiễu khác (Gaussian, Poisson) để đánh giá khả năng thích nghi trên ảnh Full HD.
- Tích hợp học sâu để tự động điều chỉnh các tham số, nâng cao hiệu suất trên các tập dữ liệu đa dạng.

REFERENCES

- [1] I. Haq, S. Anwar, K. Shah, M. T. Khan, and S. A. Shah, *Fuzzy Logic Based Edge Detection in Smooth and Noisy Clinical Images*, PLOS ONE, vol. 10, no. 9, e0138712, 2015. DOI: <https://doi.org/10.1371/journal.pone.0138712>.
- [2] I. Sobel, *An Isotropic 3x3 Gradient Operator*, Stanford Artificial Project, 1970.
- [3] J. L. Canny, *The Edge Detection Operator*, Proceedings of the IEEE, vol. 71, no. 7, pp. 1177-1184, 1983.
- [4] D. Marr, E. Hildreth, *Theory of Edge Detection*, Proceedings of the Royal Society of London, vol. 207, no. 1167, pp. 187-217, 1980.
- [5] X. Jiang, H. Bunke, *Edge Detection in Range Images*, Computer Vision and Image Understanding, vol. 73, no. 2, pp. 183-199, 1999.
- [6] C. Genming, Y. Baozong, *A New Edge Detector*, Journal of Electronics (China), vol. 6, no. 4, pp. 314-319, 1989.
- [7] D.-S. Kim et al., *Automatic Edge Detection*, Pattern Recognition Letters, vol. 25, no. 1, pp. 101-106, 2004.
- [8] E. K. Kaur et al., *Fuzzy Logic Based Edge Detection*, International Journal of Computer Applications, vol. 1, no. 22, pp. 55-58, 2010.
- [9] J. M. S. Prewitt, *Object Enhancement and Extraction*, Proceedings of the IEEE, vol. 51, no. 2, pp. 201-209, 1970.